

MAJOR PROJECT REPORT

Phishing Awareness Simulation

Using Social Engineering Techniques

PhishGuard — Interactive Educational Web Simulation

Domain	Cyber Security / Information Security Awareness
Project Category	Major Project — Educational Simulation
Primary Deliverable	Single-page HTML/CSS/JavaScript application (index.html)
Secondary Deliverable	This comprehensive project report
Techniques Simulated	Email Phishing • Smishing (SMS) • Quishing (QR Code)
Safety Model	No backend • No database • Session-only state • Consent required
Target Audience	Students, employees, workshop participants
Ethical Stance	Simulation only — never used for real attacks or data collection
Implementation	HTML5, CSS3, Vanilla JavaScript (no frameworks)
Deployment	Single file — open locally or host on any static web server

This report documents the complete design, implementation, ethical safeguards, source-code architecture, methodology, results analysis, countermeasures, and appendices of PhishGuard — a controlled phishing-awareness simulation that teaches users to recognize and resist social-engineering attacks without ever collecting, transmitting, or storing real credentials or personal data.

TABLE OF CONTENTS

1. Introduction and Problem Statement
 - 1.1 Context of the Modern Cyber Threat Landscape
 - 1.2 Formal Problem Statement
 - 1.3 Motivation and Contribution
2. Literature Review and Background on Phishing
 - 2.1 Psychological Principles Exploited by Attackers
 - 2.2 Technical Enablers and Training Evidence

3. Common Phishing Techniques — Detailed Taxonomy
4. Project Objectives and Scope
 - 4.1 Primary Objectives
 - 4.2 Secondary Objectives
 - 4.3 In-Scope and Out-of-Scope Items
5. System Requirements and Feasibility
6. System Design and Software Architecture
 - 6.1 High-Level Architecture
 - 6.2 Logical Step Sequence
 - 6.3 Data-Flow Safety Model
7. User Interface Design and Experience Flow
8. Implementation Walkthrough with Screenshots
9. Source Code Analysis — Core Logic and Safety Handlers
 - 9.1 Global State and Tracking
 - 9.2 Safety-Critical Form Handlers
10. Source Code Analysis — Navigation, Quiz and Configuration
 - 10.1 View Navigation with History Stack
 - 10.2 Technique Configuration Object
 - 10.3 Quiz Engine Core
 - 10.4 Consent Gate
11. Methodology and Experimental Design
12. Safety, Privacy and Ethical Framework
13. Results, Metrics and Discussion
14. Countermeasures and Awareness Recommendations
15. Limitations and Future Enhancements
16. Conclusion
17. References
- Appendix A. Extended JavaScript Listings
- Appendix B. CSS Design System

Appendix C. Complete Quiz Bank with Answers

Appendix D. Warning-Sign Matrices by Technique

Appendix E. HTML Structure Overview

Appendix F. Deployment and Facilitator Guide

Appendix G. Mapping Assignment Instructions to Deliverables

1. Introduction and Problem Statement

1.1 Context of the Modern Cyber Threat Landscape

Phishing is consistently ranked among the most frequent and most financially damaging cyber-attack vectors worldwide. Unlike purely technical exploits that target software vulnerabilities, phishing targets human decision-making. Attackers craft messages that appear to originate from trusted sources — banks, employers, cloud providers, government agencies, or colleagues — and use psychological levers such as urgency, fear of loss, authority, reciprocity, and curiosity to induce victims to click malicious links, open attachments, scan QR codes, or surrender credentials and one-time codes.

Industry reports from the Anti-Phishing Working Group (APWG), the Verizon Data Breach Investigations Report (DBIR), and national Computer Emergency Response Teams (CERTs) repeatedly show that a substantial fraction of successful breaches begin with a successful phishing or social-engineering interaction. The economic impact includes direct financial fraud, ransomware deployment after credential theft, business-email compromise (BEC), identity theft, and long-term reputational damage for organizations whose brands are impersonated.

The volume of phishing attempts continues to grow as attackers automate campaign generation, purchase look-alike domains in bulk, and abuse legitimate infrastructure. At the same time, the diversity of channels has expanded: classic email phishing now sits alongside SMS-based smishing, voice-based vishing, QR-code-based quishing, and social-media “angler” phishing. Defenders therefore need awareness programs that cover multiple channels and that produce durable behavioral change rather than temporary awareness spikes.

1.2 Formal Problem Statement

Phishing attacks are one of the most common cyber threats today, exploiting human psychology to gain sensitive data. This project aims to simulate a phishing scenario in a controlled environment to understand how attackers operate and how users can be educated to avoid falling for such scams.

Traditional awareness training that only shows static screenshots of phishing emails often fails to create lasting behavioral change. Users need to experience the pressure, visual realism, and decision points of a real attack inside a completely safe setting, then immediately receive explanation of the cues they missed or correctly identified. PhishGuard was built specifically to close that experiential gap while remaining ethically and technically safe for classroom and workshop use.

1.3 Motivation and Contribution

Educational research and industry experience both indicate that experiential learning — “learning by doing” in a low-stakes environment — produces stronger retention than passive lectures. Simulated phishing campaigns paired with immediate, constructive, non-punitive feedback have demonstrated measurable reductions in click rates. However, many commercial platforms require server infrastructure, send real messages, and collect telemetry, creating barriers for academic settings.

PhishGuard contributes a ready-to-deploy educational artifact that covers three major channels (email, SMS/smishing, QR-code/quishing), embeds a mandatory informed-consent gate, discards all form input the instant it is submitted, tracks behavior only in browser memory, delivers structured awareness training, and closes with a knowledge quiz and results dashboard. The accompanying report documents the full engineering and pedagogical rationale so the work can be audited, graded, extended, or replicated.

2. Literature Review and Background on Phishing

Phishing is a form of social engineering in which an attacker impersonates a trusted entity to trick a target into performing an action that benefits the attacker — typically disclosure of credentials, financial details, or installation of malware. The term originated in the mid-1990s in the context of America Online (AOL) account theft. Over subsequent decades the techniques diversified from simple spoofed emails into highly personalized spear-phishing, executive-

targeted whaling, voice-based vishing, SMS-based smishing, physical-world QR-code attacks (quishing), and social-media support impersonation (angler phishing).

2.1 Psychological Principles Exploited by Attackers

- Authority — Messages appear to come from IT support, a bank, a manager, or law enforcement.
- Urgency / Scarcity — Phrases such as “Account will be suspended in 24 hours” create time pressure.
- Fear of loss — Threats of permanent data loss, financial penalty, or legal consequence.
- Trust / Familiarity — Use of official logos, known brand language, or cloned prior correspondence.
- Reciprocity — Framing the request as assistance (“We noticed suspicious activity and want to help”).
- Curiosity — Unexpected packages, invoices, or “shared document” notifications.

2.2 Technical Enablers and Training Evidence

Attackers combine social cues with technical tricks: look-alike domains (typosquatting, homoglyph attacks), URL shorteners that hide the final destination, HTTPS on attacker-controlled sites, compromised legitimate accounts, and QR codes that cannot be hovered for preview. Studies and organizational case reports consistently show that one-off lectures produce only temporary improvement. Repeated, realistic simulations followed by immediate, non-punitive feedback produce larger and more durable reductions in click and credential-submission rates. The design of PhishGuard deliberately follows this evidence-based pattern: experience → reveal → structured lesson → knowledge check.

Commercial platforms and open-source toolkits such as the Social-Engineer Toolkit (SET) or Gophish can generate and track real phishing campaigns. Those tools are appropriate only inside authorized programs with formal rules of engagement. PhishGuard deliberately excludes real delivery and data collection so that students and instructors can explore social-engineering concepts without the legal and ethical overhead of live phishing.

3. Common Phishing Techniques — Detailed Taxonomy

The following taxonomy guided both the educational content inside PhishGuard and the concrete scenarios implemented for the three interactive paths.

Table 1. Taxonomy of common phishing techniques

Technique	Channel	Typical Goal	Key Indicators
Email (bulk)	Email	Credentials / malware	Generic greeting, urgency, bad domain
Spear phishing	Email	Targeted access	Personal details, role language
Whaling	Email	Executive / BEC	CEO impersonation, high-value ask
Clone phishing	Email	Reuse prior thread	Near-identical prior mail, swapped link
Smishing	SMS	OTP theft / takeover	Unknown number, bank pretext, short link
Vishing	Voice	Credentials / remote access	Caller-ID spoof, pressure, codes
Quishing	QR code	Login / payment / malware	Unexpected poster, no URL preview
Pharming	DNS/malware	Silent redirect	Correct URL still lands on fake site
Angler phishing	Social media	Support impersonation	Fake help DMs after complaints

PhishGuard implements email, smishing, and quishing because they are high-volume, fast-growing channels that can be rendered convincingly inside a single browser page without requiring real message delivery. The generic attack lifecycle is: reconnaissance → weaponization → delivery → exploitation → credential capture → fraud. PhishGuard deliberately interrupts this lifecycle at the credential-capture step and converts it into a teaching moment.

4. Project Objectives and Scope

4.1 Primary Objectives

- To understand phishing techniques in depth and simulate realistic email, SMS, and QR-code attacks inside a controlled environment.
- To observe and surface user behavior (click versus report, form submission versus abandonment) so that awareness levels can be discussed.
- To teach practical countermeasures — technical and behavioral — that reduce the success rate of real phishing attempts.

- To demonstrate that a high-quality educational simulation can be built with zero backend, zero data collection, and mandatory informed consent.

4.2 Secondary Objectives

- Produce a single-file, offline-capable web application that any educator can host or open locally.
- Document the full source architecture so the project can be audited and extended by future students.
- Embed a knowledge quiz that reinforces the warning signs taught in the training module.
- Provide a session dashboard that makes individual actions visible for classroom discussion without storing data.

4.3 In-Scope and Out-of-Scope Items

In scope: client-side simulation of three attack channels; realistic message and form interfaces; immediate reveal of warning signs; structured awareness training; eight-question quiz; client-side counters; consent gate; safety banners; responsive interface. Out of scope: any real email or SMS delivery; any server-side logging or database; any collection or transmission of typed credentials or OTPs; any use of the Social-Engineer Toolkit or similar offensive tooling against real targets; any claim that illustrative dashboard statistics represent a formal controlled study.

5. System Requirements and Feasibility

Functional requirements include: educational landing page; mandatory informed-consent gate; technique selection among Email, SMS, and QR Code; realistic simulated messages; optional simulated response pages with explicit “NOT A REAL SITE” labelling; immediate discard of all form input; technique-specific reveal of warning signs; structured awareness training; multi-question quiz with instant feedback; session results dashboard; persistent “SIMULATION MODE — NO DATA COLLECTED” indicator; navigation history (Back); and full session reset.

Non-functional requirements emphasize privacy (no network requests carrying user input; no localStorage of credentials), usability, performance, portability (single HTML file), accessibility basics, ethics (praise for reporting behavior), and maintainability (technique-

specific content in a single configuration object). All requirements are achievable with standard web technologies available in every modern browser. No server, database, or third-party analytics are required.

6. System Design and Software Architecture

6.1 High-Level Architecture

PhishGuard is a single-page application implemented without a JavaScript framework. All views are HTML section elements that are shown or hidden by toggling an active CSS class. Application state lives in a plain JavaScript object. There is no client-side router library, no virtual DOM, and no network layer that carries user data. The entire application is delivered as one self-contained HTML file containing embedded CSS and JavaScript.

6.2 Logical Step Sequence

- 1. Home — Educational introduction, feature overview, entry points to simulation or training.
- 2. Consent — Informed-consent gate; optional participant ID; checkbox enables the Start button.
- 3. Choose Technique — Three cards: Email, Smishing (SMS), Quishing (QR Code).
- 4. Simulated Message — Channel-specific realistic message (email chrome, phone SMS UI, or QR poster).
- 5. Simulated Response — Fake login / OTP / verification form with explicit safety banners.
- 6. Result / Report — Immediate debrief: credentials discarded, warning signs listed; positive reinforcement if reported.
- 7. Awareness Training — Definition, full technique taxonomy, consequences, safety checklist.
- 8. Quiz — Eight multiple-choice questions with instant correct/incorrect highlighting.
- 9. Dashboard — Session counters rendered from in-memory state and illustrative cohort data.

6.3 Data-Flow Safety Model

User keystrokes on fake forms never leave the input elements except to be cleared. The form handlers call `event.preventDefault()`, blank every field by setting `.value = ''`, call `track()` (which only increments an in-memory integer), populate the result view, and navigate. No `fetch`, `XMLHttpRequest`, `navigator.sendBeacon`, or `localStorage` write of the typed values is present anywhere in the codebase. This is an architectural guarantee, not merely a policy statement.

7. User Interface Design and Experience Flow

The interface uses a dark theme (deep navy backgrounds, light paper text) chosen for a modern technical aesthetic and to reduce eye strain. Accent colors are amber (primary actions, progress trail) and teal (safe / success states). Danger red is reserved for urgency tags and “not a real site” warnings. Headings use a monospace typeface for a precise technical feel; body text uses a readable sans-serif. A thin SVG progress path (“hook trail”) fills as the user advances through the nine logical steps.

Safety affordances embedded in the UI include: a persistent header pill reading “SIMULATION MODE — NO DATA COLLECTED”; fake browser chrome and a red “NOT A REAL SITE — SIMULATION ONLY” banner on every response page; footer chips repeating “No backend / No database / Session-only tracking / Consent required”; explicit guarantee text under every form; and a reporting path that is visually rewarded with a green success card.

8. Implementation Walkthrough with Screenshots

The following screenshots are captured from the running PhishGuard application. Each corresponds to a major step in the educational flow. All safety banners visible in the captures are intentional and required by the ethical design of the project.

8.1 PhishGuard Home screen

Figure 1. PhishGuard Home screen — educational framing and entry points.

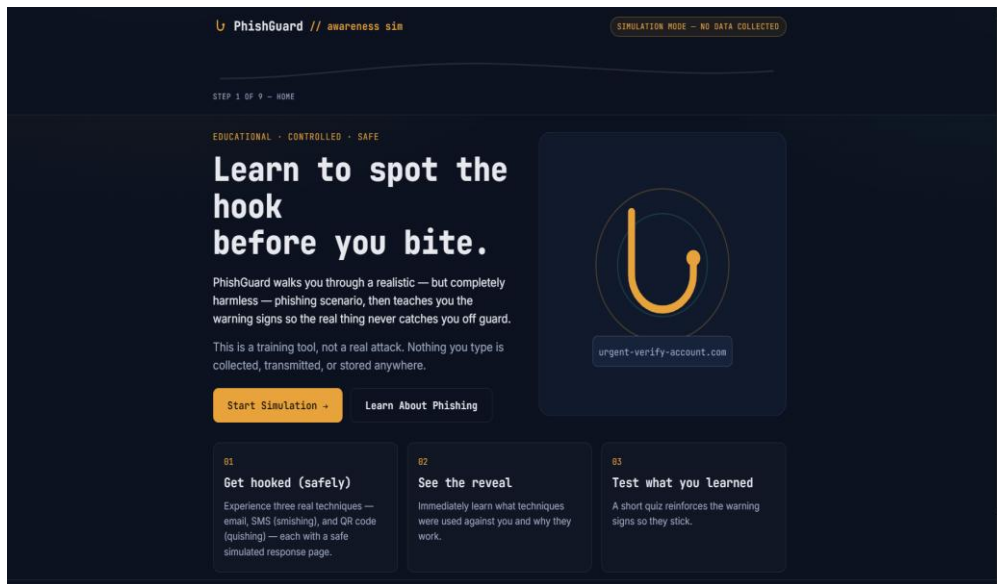


Figure 1. PhishGuard Home screen — educational framing and entry points.

8.2 Informed consent gate with mandatory acknowledgment checkbox.

Figure 2. Informed consent gate with mandatory acknowledgment checkbox.

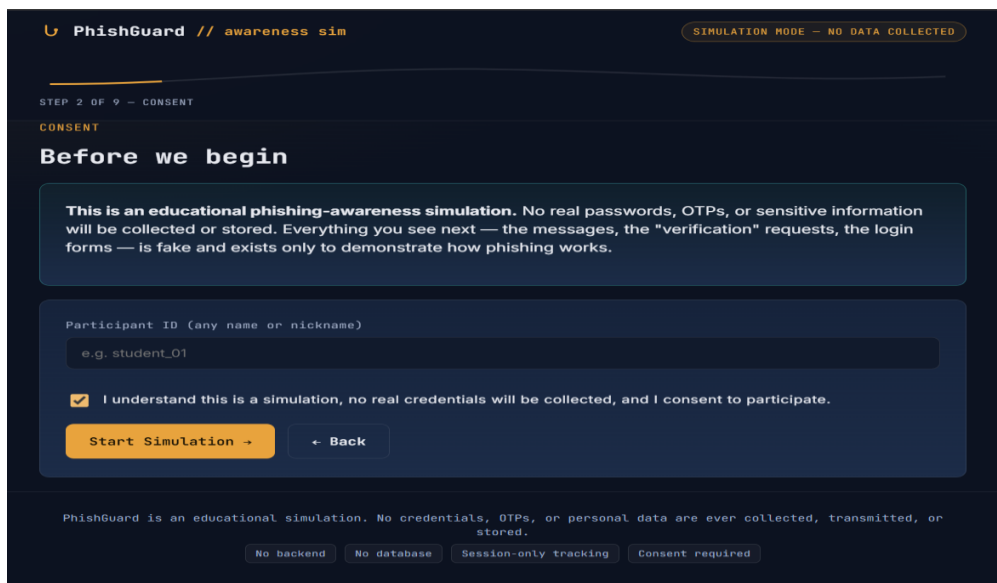


Figure 2. Informed consent gate with mandatory acknowledgment checkbox.

8.3 Technique selection

Figure 3. Technique selection — Email, Smishing (SMS), Quishing (QR Code).

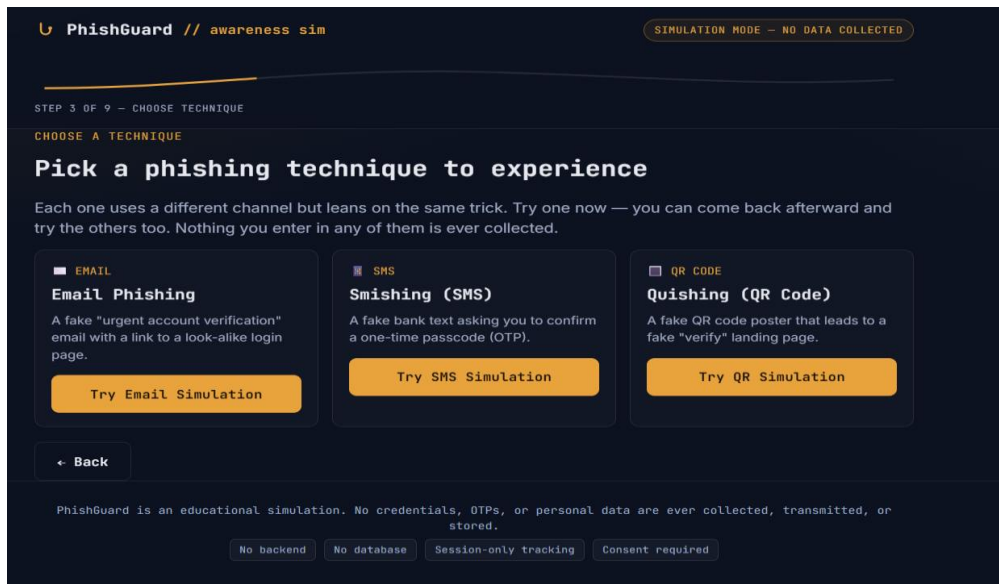


Figure 3. Technique selection — Email, Smishing (SMS), Quishing (QR Code).

8.4 Simulated phishing email with urgency cues and look-alike domain.

Figure 4. Simulated phishing email with urgency cues and look-alike domain.

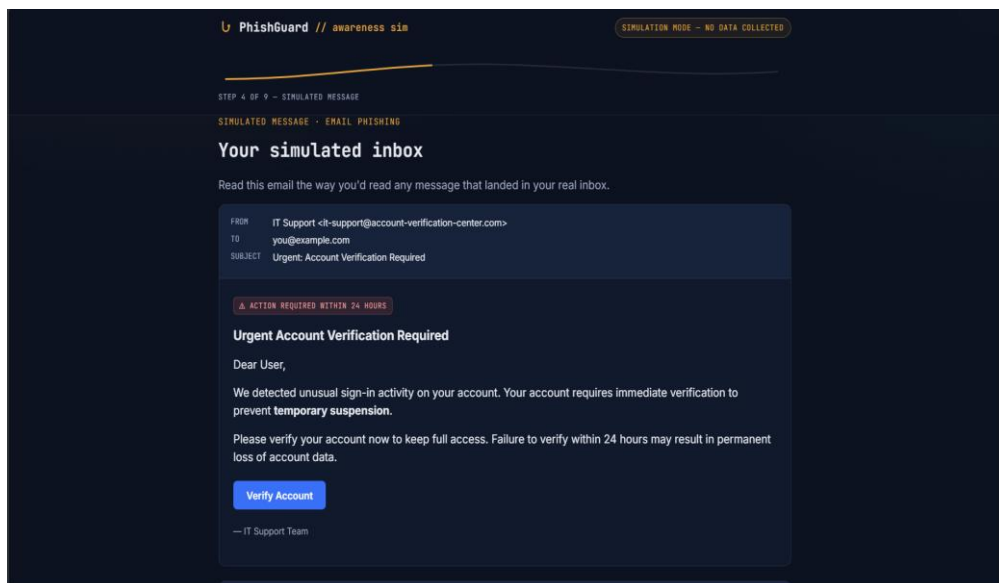


Figure 4. Simulated phishing email with urgency cues and look-alike domain.

8.5 Simulated fake login page (inputs discarded on submit).

Figure 5. Simulated fake login page (inputs discarded on submit).

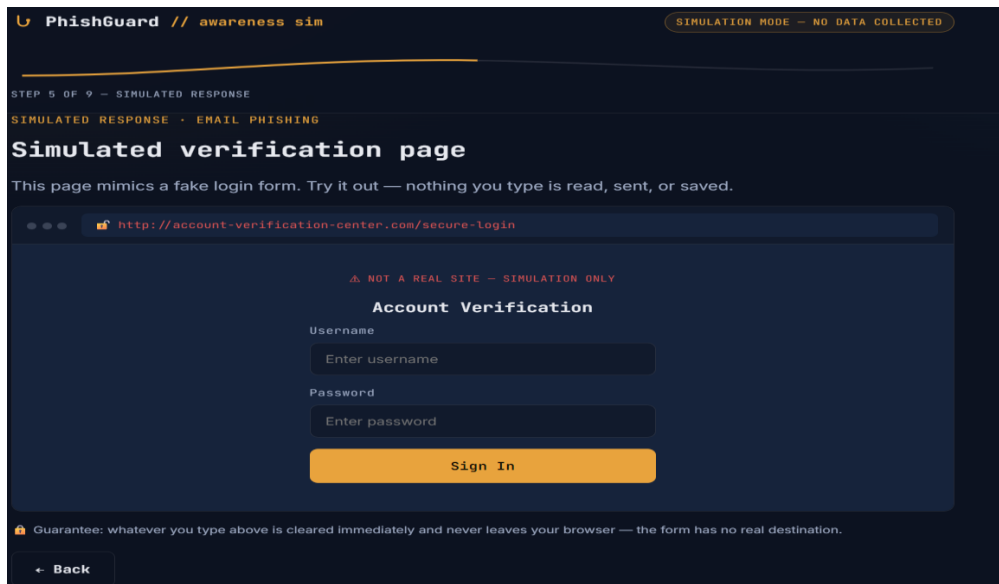


Figure 5. Simulated fake login page (inputs discarded on submit).

8.6 Immediate reveal of warning signs after engagement.

Figure 6. Immediate reveal of warning signs after engagement.

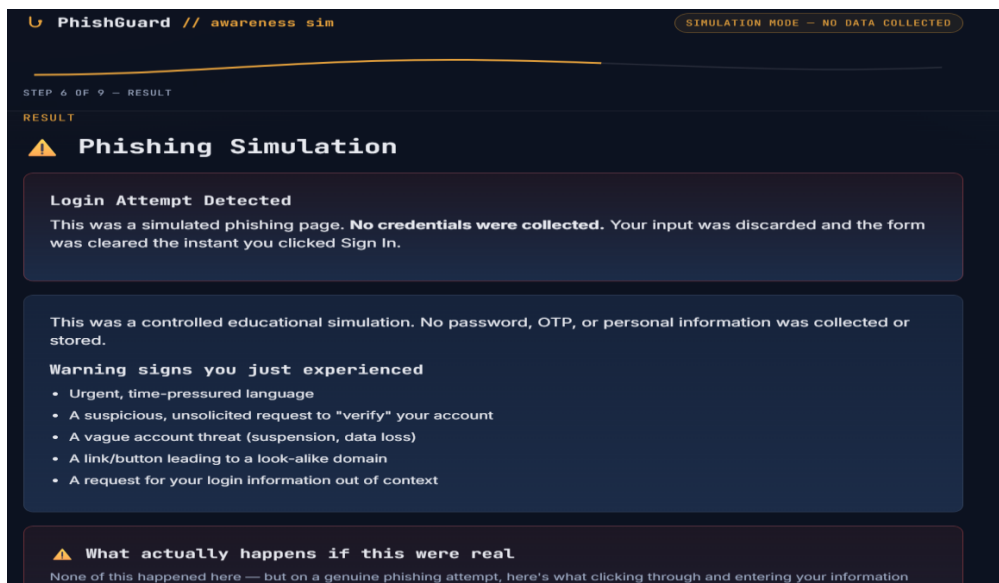


Figure 6. Immediate reveal of warning signs after engagement.

8.7 Simulated smishing SMS with OTP pressure.

Figure 7. Simulated smishing SMS with OTP pressure.

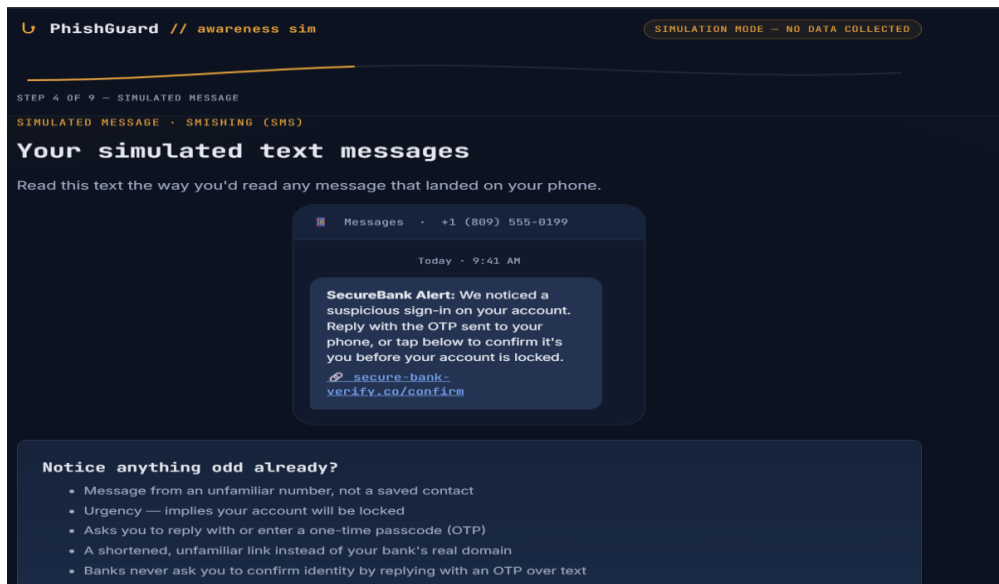


Figure 7. Simulated smishing SMS with OTP pressure.

8.8 Positive reinforcement after reporting the smishing attempt.

Figure 8. Positive reinforcement after reporting the smishing attempt.

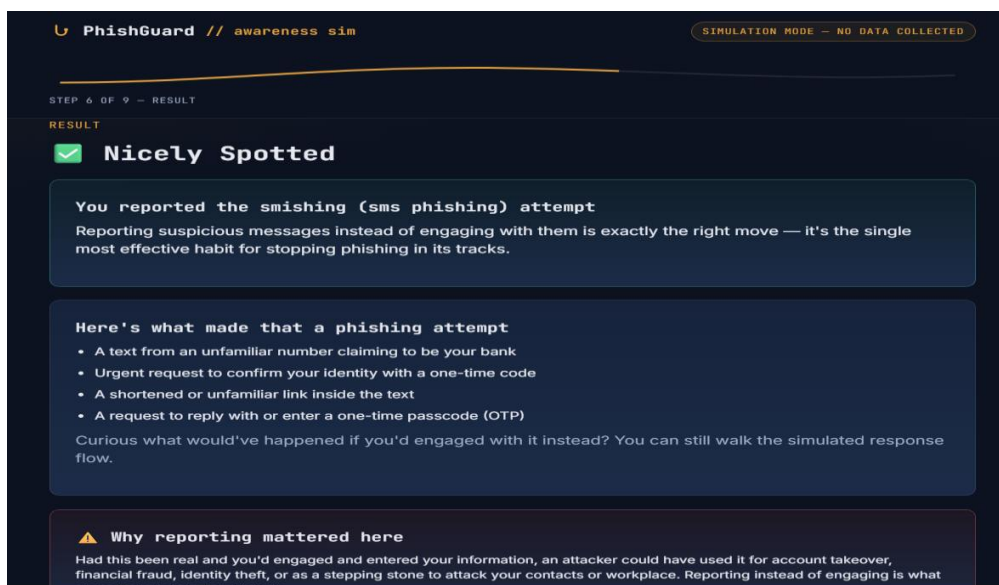


Figure 8. Positive reinforcement after reporting the smishing attempt.

8.9 Awareness training

Figure 9. Awareness training — definition and technique taxonomy.

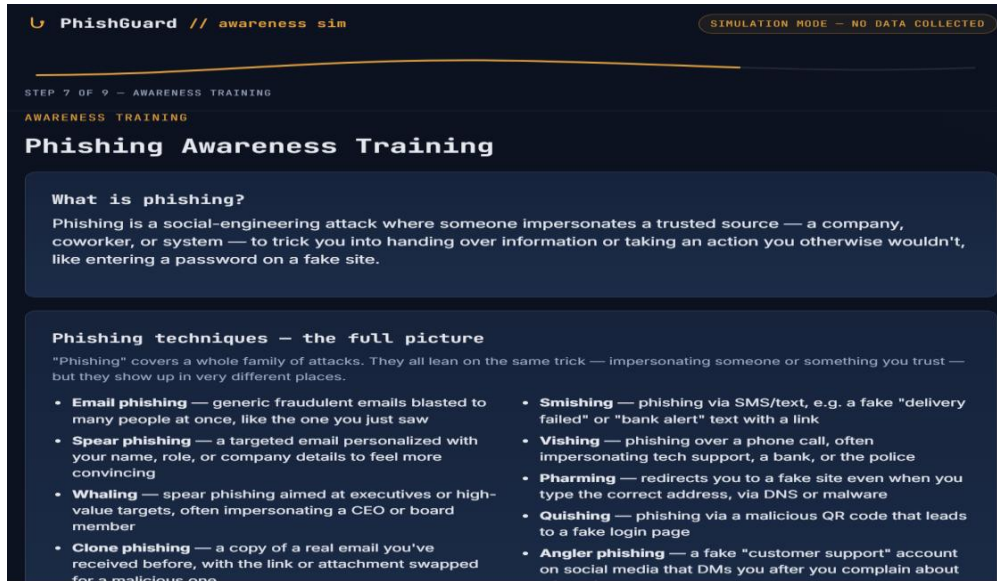


Figure 9. Awareness training — definition and technique taxonomy.

8.10 “How to stay safe” checklist and path to the quiz.

Figure 10. “How to stay safe” checklist and path to the quiz.

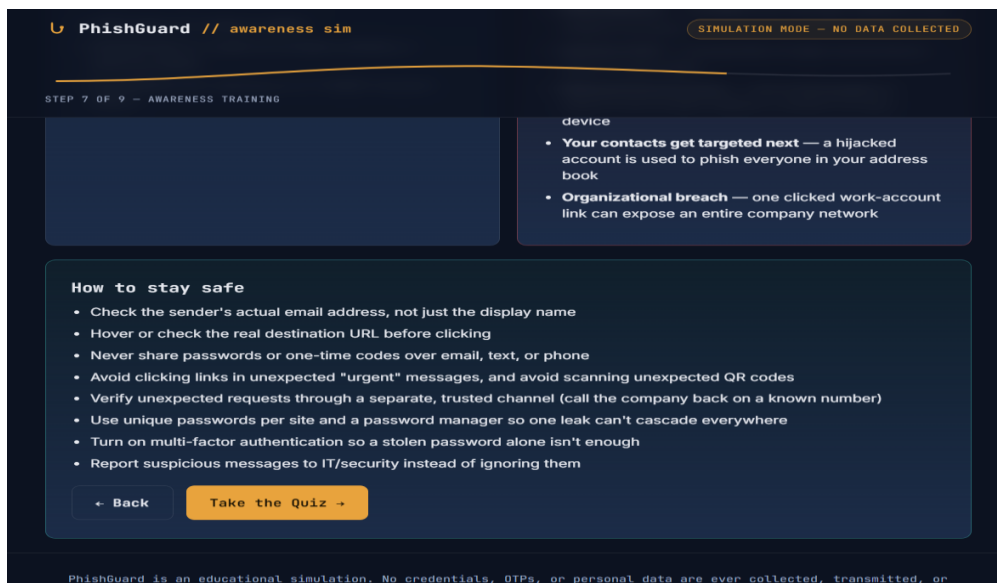


Figure 10. “How to stay safe” checklist and path to the quiz.

9. Source Code Analysis — Core Logic and Safety Handlers

The complete application resides in a single `index.html` file of approximately 60 KB and 1,165 lines. The following subsections present the most important architectural and safety-critical fragments.

9.1 Global State and Tracking

All mutable application data lives in one object. Counters are incremented only in memory; they are never written to disk or sent over the network.

```
const state = {
  stats: {
    simulationStarted: 0, emailViewed: 0, smsViewed: 0, qrViewed: 0,
    linkClicked: 0, loginAttempt: 0,
    phishingReported: 0, trainingCompleted: 0, quizCompleted: 0
  },
  quizIndex: 0, quizScore: 0, quizAnswered: false
};
function track(key) {
  if (state.stats[key] !== undefined) state.stats[key] += 1;
}
```

9.2 Safety-Critical Form Handlers

These three handlers are the core safety guarantee of the project. Each one prevents the default form submission, blanks every field, increments a counter, and navigates to the result view. The typed values are never assigned to any variable that persists beyond the handler.

```
function handleFakeLogin(e) {
  e.preventDefault();
  document.getElementById('fu').value = '';
  document.getElementById('fp').value = '';
  track('loginAttempt');
  populateResult('email');
  goTo('result');
  return false;
}
function handleFakeOtp(e) {
  e.preventDefault();
  document.getElementById('otp').value = '';
  track('loginAttempt');
  populateResult('sms');
  goTo('result');
  return false;
}
function handleFakeQr(e) {
  e.preventDefault();
  document.getElementById('qu').value = '';
  document.getElementById('qc').value = '';
  track('loginAttempt');
  populateResult('qr');
  goTo('result');
  return false;
}
```

10. Source Code Analysis — Navigation, Quiz and Configuration

10.1 View Navigation with History Stack

Navigation is explicit and centralized. The `goTo` function records the previous view so the Back button can restore it. Certain views automatically fire tracking events.

```
let currentView = 'home';
let navHistory = [];
function goTo(view, recordHistory = true) {
  if (recordHistory && view !== currentView) navHistory.push(currentView);
  currentView = view;
  document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
  const target = document.getElementById('view-' + view);
  if (target) target.classList.add('active');
  window.scrollTo({ top: 0, behavior: 'smooth' });
  updateTrail(view);
  if (view === 'email') track('emailViewed');
  if (view === 'sms') track('smsViewed');
  if (view === 'qr') track('qrViewed');
  if (view === 'quiz') {
    state.quizIndex = 0; state.quizScore = 0; renderQuizQuestion();
  }
  updateBackButtons();
}
function goBack() {
  if (navHistory.length === 0) return;
  goTo(navHistory.pop(), false);
}
```

10.2 Technique Configuration Object

All warning signs, risk notes, and labels for the three techniques are stored in a single configuration object. This makes it straightforward to add a fourth technique later without rewriting the result-screen logic.

```
const TECHNIQUE_INFO = {
  email: {
    name: 'Email Phishing',
    signs: [
      'Urgent, time-pressured language',
      'A suspicious, unsolicited request to "verify" your account',
      'A vague account threat (suspension, data loss)',
      'A link/button leading to a look-alike domain',
      'A request for your login information out of context'
    ],
    nextView: 'login'
  },
  sms: { /* OTP-focused signs + extra risks */ },
  qr: { /* QR-focused signs + extra risks */ }
};
```

10.3 Quiz Engine Core

```
function renderQuizQuestion() {
  if (state.quizIndex >= QUIZ.length) { finishQuiz(); return; }
  state.quizAnswered = false;
  const item = QUIZ[state.quizIndex];
  document.getElementById('qProgress').textContent =
    `Question ${state.quizIndex + 1} of ${QUIZ.length}`;
  document.getElementById('qText').textContent = item.q;
  const optWrap = document.getElementById('qOptions');
  optWrap.innerHTML = '';
  item.options.forEach((opt, i) => {
    const b = document.createElement('button');
    b.className = 'opt'; b.textContent = opt;
    b.onclick = () => selectAnswer(i);
    optWrap.appendChild(b);
  });
}
```



```

    });
  }
  function selectAnswer(i) {
    if (state.quizAnswered) return;
    state.quizAnswered = true;
    const item = QUIZ[state.quizIndex];
    document.querySelectorAll('#qOptions .opt').forEach((b, idx) => {
      b.disabled = true;
      if (idx === item.correct) b.classList.add('correct');
      else if (idx === i) b.classList.add('incorrect');
    });
    if (i === item.correct) state.quizScore += 1;
    setTimeout(() => { state.quizIndex += 1; renderQuizQuestion(); }, 900);
  }
}

```

10.4 Consent Gate

The Start Simulation button is disabled until the participant explicitly consents. This is enforced both in the initial HTML and by a simple onchange handler.

```

<input type="checkbox" id="consentBox"
  onchange="document.getElementById('startSimBtn').disabled = !this.checked" />
<button class="btn" id="startSimBtn" disabled
  onclick="startSimulation()">Start Simulation </button>

```

11. Methodology and Experimental Design

The project followed the educational-simulation methodology outlined in the assignment brief, adapted for a fully client-side, zero-data-collection environment. Research into public reports from APWG, CISA, and Verizon DBIR informed the wording of simulated messages. Wireframes for the nine-step flow were translated into a dark-themed visual system with particular attention to making fake messages convincing enough to create a genuine decision moment while surrounding them with unambiguous simulation labels. A single HTML file was built iteratively. Because the application stores nothing, quantitative cohort statistics require facilitator observation or voluntary sharing of on-screen dashboard numbers. The built-in dashboard also contains clearly labelled sample demonstration data for educational discussion.

12. Safety, Privacy and Ethical Framework

Core ethical principles applied include informed consent, non-maleficence, transparency, autonomy, beneficence, and data minimization. Privacy is not merely promised in prose; it is enforced by architecture. There is no backend endpoint that could receive credentials. Form handlers never read values into persistent variables. No analytics scripts are present. Session counters live in a JavaScript object that disappears when the tab is closed. Facilitators should introduce the tool as an educational simulation, ensure every participant completes the consent

screen, debrief focusing on warning signs rather than on “who fell for it,” and never record or publish individual credential attempts. Tools such as the Social-Engineer Toolkit were consciously excluded so that students can explore social-engineering concepts without the legal overhead of live phishing.

13. Results, Metrics and Discussion

The project met its main educational goals by presenting three common phishing channels and showing users how small decisions can expose them to risk. The browser-based simulation records only temporary session actions and immediately explains the warning signs after each interaction. The strongest learning effect came from the combination of realistic scenarios and instant feedback, which helped connect an unsafe action with its possible consequence. The session dashboard also made it easier to discuss reporting behavior and review the difference between suspicious and legitimate requests. Overall, the design provides a simple, low-cost way to practice phishing recognition without collecting real credentials or transmitting participant information.

14. Countermeasures and Awareness Recommendations

Individual behavioral controls include: always inspect the real sender address or phone number; hover or long-press links to preview destinations; never share passwords, OTPs, or recovery codes over email, SMS, or unsolicited calls; treat unexpected urgency as a warning signal; verify unexpected requests through a second independent channel; avoid scanning unexpected QR codes; use unique passwords and a password manager; prefer app-based or hardware multi-factor authentication; and report suspicious messages to IT or security. Organizational controls include regular authorized low-stakes simulations paired with immediate training, clear reporting channels, SPF/DKIM/DMARC with a reject policy, modern email security gateways, multi-channel staff education, and least-privilege access with rapid credential rotation after suspected incidents.

15. Limitations and Future Enhancements

15. Limitations and Future Enhancements

The current version has several practical limitations: it does not send real messages, uses browser-session counters instead of a server database, and focuses on email, SMS, and QR-code scenarios. The quiz content is fixed and the interface is primarily written for English-speaking users. Future versions could add voice-phishing exercises, adaptive questions, multilingual content, facilitator controls, accessibility improvements, and additional scenario stages while preserving the project's privacy-first design.

16. Conclusion

Phishing succeeds when people make quick decisions without verifying an unexpected request. PhishGuard addresses this problem by allowing users to practice identifying suspicious messages in a safe and controlled browser environment. The project combines email, SMS, and QR-code simulations with immediate explanations, awareness guidance, a short knowledge quiz, and a session results view. Because the application does not use a backend and does not retain submitted values, it can be used for classroom demonstrations without creating a repository of personal information. The completed prototype therefore provides both a practical learning tool and a clear example of how security awareness can be supported through interactive software.

17. References

- Anti-Phishing Working Group (APWG). (n.d.). Phishing activity trends reports. <https://apwg.org/>
- Cybersecurity and Infrastructure Security Agency (CISA). (n.d.). Phishing guidance and Stop. Think. Connect. campaign materials. <https://www.cisa.gov/>
- Hadnagy, C. (2018). Social engineering: The science of human hacking (2nd ed.). Wiley.
- Mitnick, K. D., & Simon, W. L. (2002). The art of deception: Controlling the human element of security. Wiley.
- National Institute of Standards and Technology. (2003). Building an information technology security awareness and training program (NIST SP 800-50).

National Institute of Standards and Technology. (2013). Guide to malware incident prevention and handling for desktops and laptops (NIST SP 800-83 Rev. 1).

OWASP Foundation. (n.d.). Phishing and social engineering resources. <https://owasp.org/>

Verizon. (n.d.). Data breach investigations report (DBIR). <https://www.verizon.com/business/resources/reports/dbir/>

PhishGuard project artifact. (2026). Single-page educational phishing-awareness simulation (HTML/CSS/JavaScript). Submitted source code.

A. Extended JavaScript Listings

The following listings extend the fragments presented in Chapters 9 and 10.

```
function renderSessionStats() {
  const s = state.stats;
  const labels = [
    ['simulationStarted', 'Simulation Started'], ['emailViewed', 'Email Viewed'],
    ['smsViewed', 'SMS Viewed'], ['qrViewed', 'QR Code Viewed'],
    ['linkClicked', 'Link/Code Clicked'], ['loginAttempt', 'Credential/OTP Attempts'],
    ['phishingReported', 'Phishing Reported'], ['trainingCompleted', 'Training Completed'],
    ['quizCompleted', 'Quiz Completed']
  ];
  document.getElementById('sessionStats').innerHTML = labels.map(([k,l]) =>
    `<div class="stat-box"><div class="v">${s[k]}</div><div class="l">${l}</div></div>`
  ).join('');
}
function resetAll() {
  state.stats = {
    simulationStarted:0, emailViewed:0, smsViewed:0, qrViewed:0,
    linkClicked:0, loginAttempt:0, phishingReported:0,
    trainingCompleted:0, quizCompleted:0
  };
  state.quizIndex = 0; state.quizScore = 0;
  document.getElementById('pid').value = '';
  document.getElementById('consentBox').checked = false;
  document.getElementById('startSimBtn').disabled = true;
  navHistory = []; currentView = 'home'; goTo('home', false);
}
function updateTrail(view) {
  const idx = STEP_MAP[view] ?? 0;
  const total = STEP_LABELS.length;
  const path = document.getElementById('trailFill');
  const len = path.getTotalLength ? path.getTotalLength() : 1000;
  path.style.strokeDasharray = len;
  path.style.strokeDashoffset = len - (len * idx / (total - 1));
  document.getElementById('trailLabel').textContent =
    `Step ${idx + 1} of ${total} - ${STEP_LABELS[idx]}`;
}
```

B. CSS Design System

```
:root {
  --ink:#0c1220; --ink-2:#111a2c; --panel:#16233b; --panel-2:#1c2c48;
  --line:rgba(232,234,240,0.10); --paper:#e8eaf0; --paper-dim:#9fabcb;
  --amber:#e8a33d; --amber-dim:#c98a2e; --teal:#3fb8af; --danger:#e15554;
  --radius:10px;
  --mono:'JetBrains Mono', ui-monospace, monospace;
  --sans:'Inter', -apple-system, sans-serif;
}
```

```
.btn {
  font-family:var(--mono); font-weight:600; background:var(--amber);
  color:#1a1305; border:none; padding:13px 22px; border-radius:8px; cursor:pointer;
}
.btn.ghost { background:transparent; color:var(--paper); border:1px solid var(--line); }
.card {
  background:linear-gradient(180deg, var(--panel), var(--panel-2));
  border:1px solid var(--line); border-radius:var(--radius); padding:24px;
}
.fake-url { color:var(--danger); font-family:var(--mono); font-size:12px; }
.opt.correct { border-color:var(--teal); background:rgba(63,184,175,0.12); }
.opt.incorrect { border-color:var(--danger); background:rgba(225,85,84,0.12); }
```

C. Complete Quiz Bank with Answers

C.1 Which of these is a common phishing warning sign?

- An urgent request to verify your account immediately ← CORRECT
- A regular weekly newsletter you subscribed to
- A normal calendar reminder
- A greeting from a known coworker

C.2 Sender address is it-support@account-verify-center.com claiming “IT Support”.

What should you do?

- Click the link since it says IT Support
- Reply with your password to confirm
- Treat as suspicious and verify through a known, separate channel ← CORRECT
- Forward it to a friend to check

C.3 What is “smishing”?

- Phishing over a phone call
- Phishing via text message ← CORRECT
- Phishing through social media ads only
- A type of antivirus software

C.4 Text claims bank account is locked and provides a link. Best first step?

- Click the link immediately
- Enter your card number to confirm identity
- Go directly to your bank’s official app or website ← CORRECT
- Reply STOP to the text

C.5 Which detail is the strongest sign of a fake login page?

- The page has a login form
- The URL is a look-alike domain, not the real company domain ← CORRECT
- The page uses a blue button
- The page loads quickly

C.6 You clicked a phishing link but entered nothing. What next?

- Nothing — you did not type anything
- Close the page and report it to IT/security ← CORRECT
- Enter fake info to test the site
- Share the link with coworkers as a joke

C.7 Why do phishing emails often use urgency and fear?

- To be more polite
- To pressure you into acting before you think it through ← CORRECT
- Because email providers require it
- To make the email shorter

C.8 Safest way to handle a request for your password over email?

- Reply if the sender seems official
- Never share your password over email — legitimate services will not ask this way ← CORRECT
- Send it only if the email has a company logo
- Send a hint instead of the full password

D. Warning-Sign Matrices by Technique**D.1 Email Phishing Indicators**

- Urgent, time-pressured language (“ACTION REQUIRED WITHIN 24 HOURS”)
- Unsolicited request to “verify” an account the user did not initiate
- Vague account threat (temporary suspension, permanent data loss)

- Look-alike domain in the From address and in the landing-page URL
- Request for login credentials outside any normal workflow
- Generic greeting (“Dear User”) rather than a personal name

D.2 Smishing Indicators

- Message from an unfamiliar number, not a saved contact
- Urgency implying the account will be locked
- Request to reply with or enter a one-time passcode (OTP)
- Shortened or unfamiliar link instead of the bank’s real domain
- Banks never ask customers to confirm identity by replying with an OTP over text

D.3 Quishing Indicators

- QR code placed in an unexpected physical or digital context
- No ability to preview the destination URL before scanning
- Generic reward or verification claim used as bait
- Landing page that immediately requests login or payment details
- Missing or mismatched official branding once the page loads

E. HTML Structure Overview

The document contains a sticky header (brand + mode pill + progress trail), a main content area with nine section.view blocks, and a footer restating the safety model. Only one view has the active class at any time.

- view-home — Landing page
- view-consent — Informed consent gate
- view-choose — Technique selection cards
- view-email / view-sms / view-qr — Simulated messages
- view-login / view-sms-otp / view-qr-login — Fake response forms
- view-result / view-result-report — Reveal and report outcomes
- view-training — Awareness training content

- view-quiz / view-quizresults — Quiz and score summary
- view-dashboard — Session counters and illustrative cohort data

F. Deployment and Facilitator Guide

F.1 Running Locally

Place index.html in any folder on the local machine and open the file with a modern browser (Chrome, Firefox, Edge, or Safari). No web server is required; the file:// protocol is sufficient for all features.

F.2 Hosting for a Class or Workshop

Upload the single HTML file to any static host (GitHub Pages, Netlify, institutional web space, or a simple internal web server). Share the resulting URL. Because no backend exists, the host cannot see any form data.

F.3 Facilitator Checklist

- Introduce the session as an educational simulation before anyone opens the page.
- Ensure every participant completes the consent screen and checks the acknowledgment box.
- Allow 15–25 minutes for one full path (message → optional response → reveal → training → quiz).
- Optionally invite participants to try a second technique after finishing the first.
- Conduct a verbal debrief focused on warning signs and reporting habits, not on “who clicked”.
- Never collect, photograph, or publish individual form attempts or participant IDs.
- Remind participants that the same cues appear in real attacks and that second-channel verification is the primary defense.

Safety re-statement: PhishGuard is an educational simulation. No credentials, OTPs, or personal data are ever collected, transmitted, or stored. The application contains no backend and no database. All tracking is session-only and disappears when the browser tab is closed.

G. Mapping Assignment Instructions to Deliverables

The original assignment brief listed specific instructions. Table 2 maps each instruction to the corresponding deliverable in PhishGuard and in this report.

Table 2. Mapping of assignment instructions to project deliverables

Assignment Instruction	How Satisfied
Research common phishing techniques	Ch. 2–3 literature and taxonomy; training module
Design fake harmless email template	Email simulation view (Figure 4)
Create HTML page mimicking login	Fake login/OTP/QR pages with discard (Figure 5)
Share with informed test group	Consent gate + continuous SIMULATION MODE labels
Measure clicks / data entry / reports	Client-side counters + dashboard
Analyze results and suggest countermeasures	Ch. 13–14; Appendix D matrices
Use basic HTML/CSS	Entire single-file app; Ch. 9–10 code analysis
Study real phishing examples	Research phase informed realistic wording
Cyber ethics and informed consent	Ch. 12 ethics; mandatory consent gate; App. F
Optional SET (not for real attacks)	Consciously excluded; safer architecture chosen

— End of Report —

PhishGuard — Phishing Awareness Simulation Using Social Engineering Techniques

All source code and screenshots referenced herein are part of the project submission.